

Industrial Tool

Git & Github

Git Remote :-

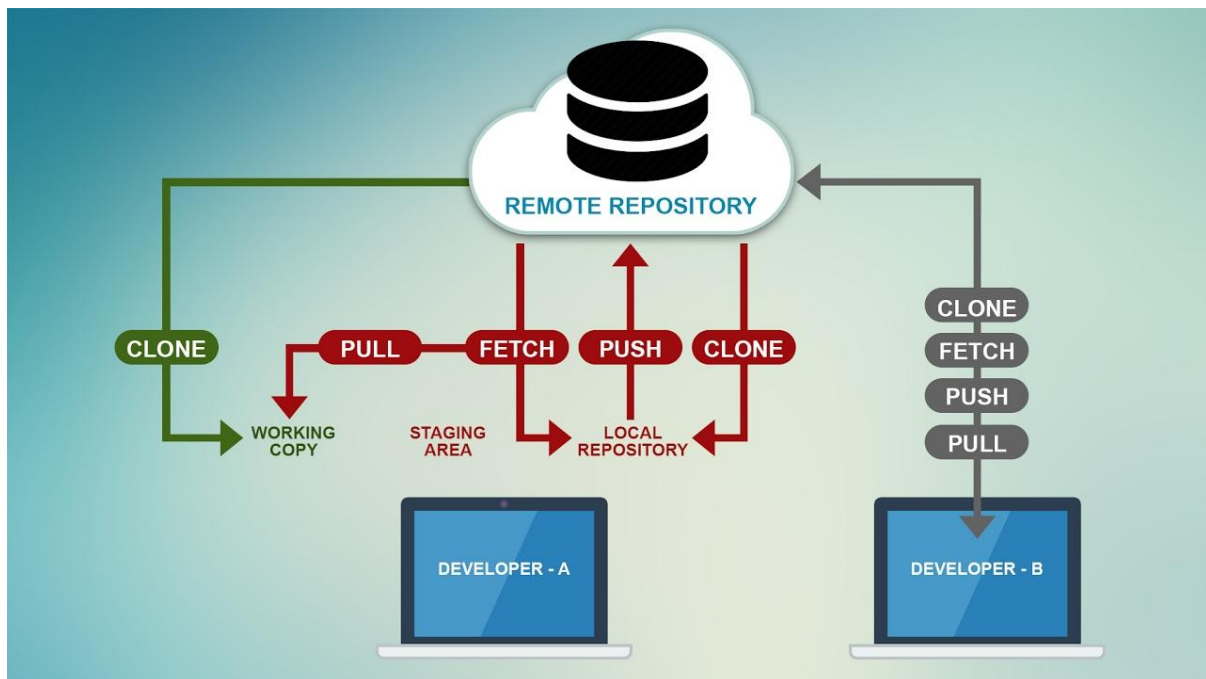
- A Git remote refers to a repository hosted on GitHub, GitLab, or a server.
- It connects your local repository to a remote repository.
- It enables sharing and synchronization of code.
 1. Push your local changes to the remote repository.
 2. Pull updates from the remote repository to your local one.

Git Origin :-

- origin is the default name for the remote repository URL.
- It points to where your code is stored (GitHub/GitLab).
- It acts like a shortcut for the remote location.
 1. origin is the key (the name),
 2. and the remote URL is the value (the actual address).

git push origin main

This tells Git to push your local main branch to the remote repository named origin.



Managing Git Remote Repositories :-

Git remote commands help you connect, manage, and interact with remote repositories like GitHub or GitLab. Here are the commonly used commands

1. Add a Remote Repository

To establish a connection between your local repository and a remote repository, use the following command:

git remote add origin <URL>

```
nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote add origin https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git
>>
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote -v
origin https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git (fetch)
origin https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git (push)
```

Add a Remote Repository

2. View Remote Repositories

To display a list of linked remote repositories along with their URLs for the local system user attempting to connect to a GitHub server.

git remote -v

```
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote -v
origin https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git (fetch)
origin https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git (push)
```

View Remote Repositories

3. Rename a Remote

To change the name of an existing remote that you are currently connected to on the server.

git remote rename <oldname> <newname>

4. Remove a Remote

To unlink a remote repository from your local repo use the following command. If the repository name does not exist you can write origin instead of the repository name.

git remote remove <repositoryname>

```
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote remove hello
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote -v
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg>
```

Remove a Remote

5. Show Remote Details

To display information about a specific remote, including branches and fetch/push URLs use the following command.

git remote show <name>

```
PS C:\Users\GFG0569\Pictures\Screenshots\codinggfg> git remote show origin
* remote origin
  Fetch URL: https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git
  Push URL: https://github.com/pranjal-dennis-nanda/ticket-maker-with-local-storage-.git
  HEAD branch: main
  Remote branch:
    main new (next fetch will store in remotes/origin)
```

Show Remote Details

Important Git Remote Commands :-

Below are some of the most useful Git remote commands that developers commonly use in real projects:

Remote Setup and Management :-

Command	Description
git remote add <name> <url>	Adds a new remote repository (e.g., origin)
git remote -v	Shows all remotes and their fetch/push URLs
git remote rename <old> <new>	Renames an existing remote
git remote remove <name>	Removes a remote from the local repository
git remote show <name>	Shows detailed info about a remote (branches, URLs, etc.)
git remote set-url <name> <new-url>	Updates the URL of an existing remote

Working with Remotes :-

Command	Description
git clone <url>	Clones a remote repository to your local machine
git fetch <remote>	Fetches changes from the remote without merging
git pull <remote> <branch>	Fetches and merges changes from a remote branch
git push <remote> <branch>	Pushes your local branch to a remote branch
git push --set-upstream origin <branch>	Sets upstream for tracking and pushes branch to remote

Remote Branch Handling :-

Command	Description
git branch -r	Lists all remote-tracking branches
git checkout --track origin/<branch>	Creates a local branch tracking a remote branch
git push origin --delete <branch>	Deletes a branch from the remote repository
git remote prune origin	Removes references to deleted branches from the remote

Git Branch :-

Introduction of Git Branch :-

- A Git branch is a separate workspace to work on new changes.
- It does not disturb the main project.
- The main code is stored in the main or master branch.
- After completing the work, changes can be merged back into the main project.

Branches make it easy to:

- Manage different tasks or features independently
- Test changes without affecting live code
- Collaborate with others efficiently

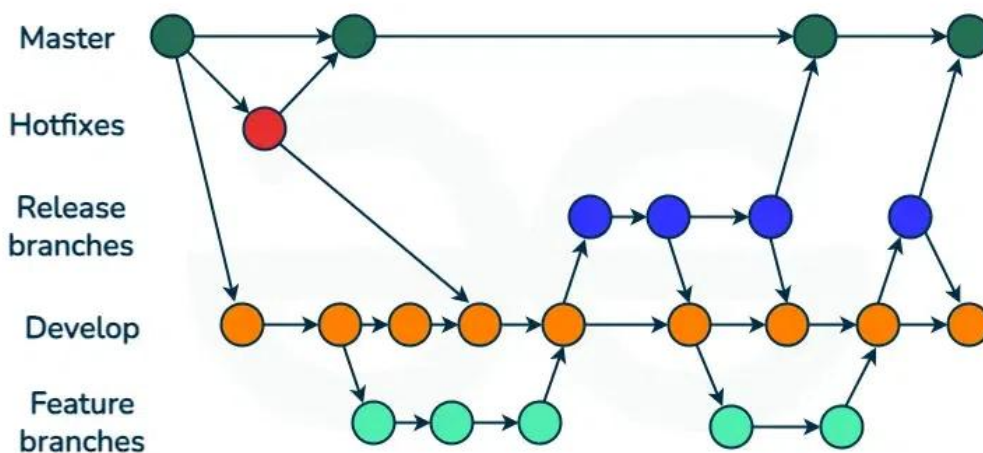
How Git Branches Work?

Git branches allow you to manage different tasks or features in isolation without affecting the main project. Here's how it works:

1. **Main Branch:** Primary branch with stable, deployable code.
2. **Feature Branch:** New branch for features or bug fixes, separate from main.
3. **Commits:** Record snapshots of changes on your branch.
4. **Merging:** Integrate completed work back into the main branch.

Types of Branches in Git :-

Git branches help organize work and let you develop features or fixes without affecting the main project.



1. **Main/Master:** Stores stable, production-ready code.
2. **Feature:** For developing new features; merged into main or develop.
3. **Develop:** Combines new features for testing before main.
4. **Hotfix:** Quick fixes for urgent production issues; merged into main.
5. **Release:** Prepares new version releases; only final fixes done.
6. **Bugfix:** Fixes specific bugs; merged into develop or main.

Step-by-Step Implementation of Creating a Branch :-

The following are the steps for creating a branch in Git:

Step 1: Create a Branch

Create a branch with the name you want to specify. Here, we are naming the branch name as "new-feature".

```
git branch new-feature
```

Step 2: Navigate to Branch

Now navigate to the new feature branch from the current branch with the following command:

```
git checkout new-feature
```

or

we can use the following command that will create the branch and navigating to the branch at a time:

```
git checkout -b new-feature
```

Step 3: Check Current Branch

Execute the following command to check the current branch that you are currently on.

```
git branch
```

Step 4: Delete a Branch

Ensure you are present on the branch you want to delete. You can delete a branch using:

```
git branch -d <branch-to-delete>
```

Basic Git Branching Commands :-

Below are some essential Git branching commands that everyone should learn when starting to work with Git branching:

1. git branch

Lists all local branches or creates a new branch when used with a name means it displays a list of all local branches or creates a new branch if a name is specified.

```
C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch
* main
```

git branch

2. git branch <branch-name>

Creates a new branch with the specified name without switching to it, allowing you to stay on your current branch.

```
: \Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch pranjal

: \Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch
main
pranjal
```

git branch <branch-name>

3. git checkout <branch-name>

Switches to an existing branch in the repository, allowing you to continue working from where it was last updated.

```
C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git checkout pranjal
D      pixel art maker
Switched to branch 'pranjal'

C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch
main
* pranjal
```

git checkout <branch-name>

4. git checkout -b <branch-name>

Creates a new branch with the specified name and immediately switches to it, allowing you to start working on it right away.

```
C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git checkout -b vipin
Switched to a new branch 'vipin'

C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch
  main
  pranjali
* vipin
```

git checkout -b <branch-name>

5. git switch <branch-name>

A modern alternative to the checkout command, primarily used for switching branches more efficiently and intuitively.

```
C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git switch pranjali
D pixel art maker
Switched to branch 'pranjali'

C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch
  main
* pranjali
  vipin
```

git switch <branch-name>

6. git switch -c <branch-name>

Creates a new branch with the specified name and switches to it immediately, allowing you to start making changes right away.

```
C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git switch -c gfg
Switched to a new branch 'gfg'

C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch
* gfg
  main
  pranjali
  vipin
```

git switch -c <branch-name>

7. git rebase <branch-name>

Puts your changes on top of another branch like in the below example as you have rebased all the changes in the feature branch are merged at the end of the main branch.

Before rebase

```
main:  A --- B --- C
      |
feature: D --- E
```

After rebase

```
main:  A --- B --- C
      |
feature: D' --- E'
```

```
C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git commit -m feature
[pranjal aee3cc9] feature
1 file changed, 116 deletions(-)
delete mode 100644 pixel art maker

C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git rebase main
Current branch pranjal is up to date.

C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>
```

git rebase <branch-name>

8. git branch -d <branch-name>

Deletes the specified branch, but only if it has been fully merged into its upstream branch, ensuring no unmerged changes are lost.

```
C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch -d vipin
Deleted branch vipin (was 54255ed).

C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch
  gfg
  main
* pranjal
```

git branch -d <branch-name>

9. git branch -D <branch-name>

Forcefully deletes the specified branch, even if it contains unmerged changes or has not been fully updated, potentially leading to data loss.

```
C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch -D gfg
Deleted branch gfg (was 54255ed).

C:\Users\GFG0569\Pictures\Screenshots\codes\Pixel-art-editor>git branch
  main
* pranjal
```

git branch -D